OSDL FINAL PROJECT REPORT

Hotel Management System

Name: Sanjay S

Reg No.: 240905110

Section: CSE-D

Roll No.: 16

1. Project Overview

The Hotel Management System is a desktop-based Java application designed to simplify and automate daily hotel operations. It is built on a strong Object-Oriented foundation, utilizing the Java Collections Framework to efficiently manage room inventory and guest records. To ensure reliability, the backend includes thread-safe mechanisms that prevent double-booking, along with file-based data serialization to securely save and reload information between sessions. On the frontend, the system features a **JavaFX Graphical User Interface (GUI)**. This interface features smooth page animations, an easy-to-use sidebar menu, and live operational statistics, making it a practical and user-friendly tool for hotel staff

2. Architectural Design Overview

This section describes the internal structure and design patterns implemented in the source code of the Hotel Management Application. The system utilizes a clear **separation of concerns** strategy, organizing code into logical units or modules (models, services, and user interface). This enhances modularity, maintainability, and reusability.

- **Models:** Represent the core data entities and data structures within the system. They hold data and state but typically do not contain complex business logic.
- **Services:** Implement the core business logic of the application. They manipulate models, manage data access, and perform complex operations (e.g., booking management, file handling).

- **User Interface (UI):** Contains classes responsible for presenting information to the user and handling user interactions.

3. Component/Class Level Description

3.1 Models Module (src/models)

This package contains POJOs (Plain Old Java Objects) that represent the business objects of the system.

Class Name	Functionality / Use	Key Methods & Attributes (Expected)
BookingPair.java	Used as a data structure to pair a Booking event with its corresponding Room or Customer. Often used in search results or to represent a full booking context.	Attributes: Room, Customer, BookingDate. Methods: getRoom(), getCustomer(), getDates().
Customer.java	Defines the data model for a hotel customer. It stores personal information and contact details.	Attributes: customerID, name, email, phone, address. Methods: getCustomerID(), setName(), updateContact().
Room.java	An abstract or base class representing a general room in the hotel. It defines common properties for all room types.	Attributes: roomNumber, isAvailable, RoomType. Methods: getRoomNumber(), checkAvailability(),

Class Name	Functionality / Use	Key Methods & Attributes (Expected)
		setAvailability(), abstract getBasePrice().
DeluxeRoom.java	Extends Room. Represents a specific type of room with extra amenities and a higher price point.	Attributes: hasBalcony, hasMiniBar. Methods: getAmenities(), calculateDeluxePrice().
StandardRoom.java	Extends Room. Represents a basic, standard room offering at the hotel.	Attributes: bedSize. Methods: getBasePrice().
RoomType.java	An Enumeration (Enum) or a class defining the various classifications of rooms (e.g., SINGLE, DOUBLE, SUITE, DELUXE).	Constants/Attributes: SINGLE, DOUBLE, DELUXE. Methods: values(), toString().

3.2 Services Module (src/services)

This package houses the classes that handle the application's functionality and connect the UI to the data/models.

Class Name	Functionality / Use	Key Methods & Responsibilities (Expected)
HotelManager.java	The central service class for all hotel operations. It acts as a	Methods: addRoom(), searchAvailableRooms(),

Class Name	Functionality / Use	Key Methods & Responsibilities (Expected)
	controller to manage rooms, bookings, and customer interactions.	makeBooking(), checkOutCustomer(), getCustomerDetails(), generateReports().
BookingTask.java	A service class specifically focused on the lifecycle of a booking. It may handle scheduling, cancellation, or background tasks related to bookings.	Methods: confirmBooking(Room, Customer), cancelBooking(BookingID), isRoomBooked(RoomID, Dates).
FileManager.java	Handles data persistence and retrieval. It likely reads and writes the system's state to external files (like .dat files).	Methods: saveCustomerData(List<Customer>), loadRoomData(), backupDatabase(), serializeSystemState().

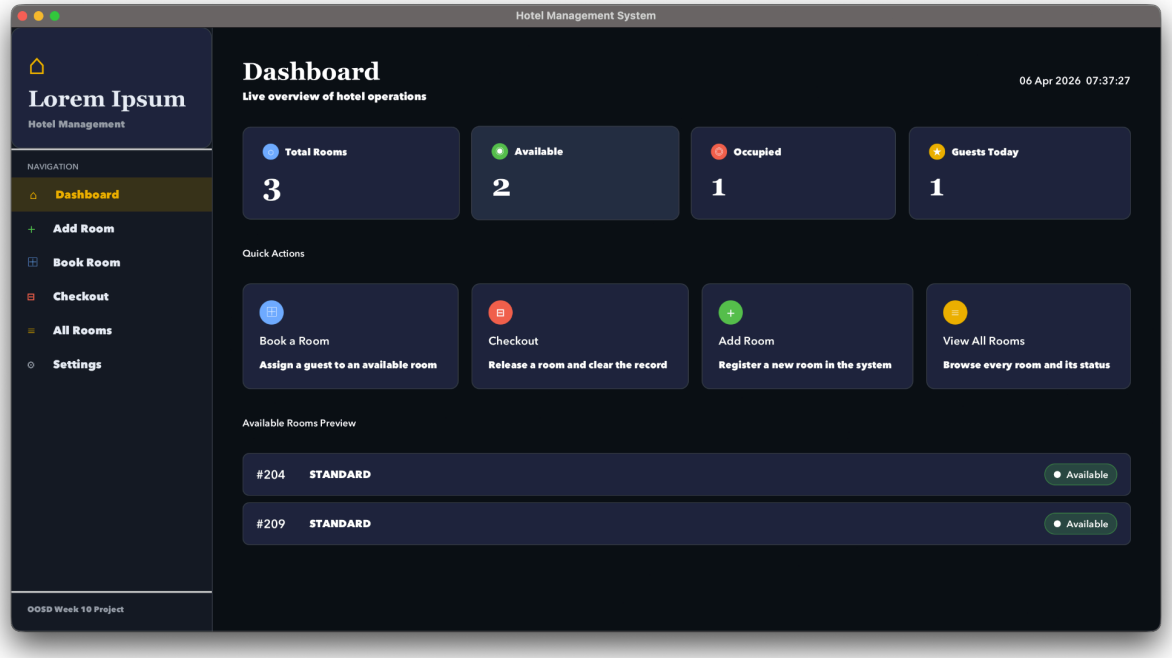
3.3 User Interface Module (src/ui)

This package contains classes that render the graphical or command-line interface for the user.

Class Name	Functionality / Use	Key Methods & Responsibilities (Expected)
MainApp.java	The entry point of the application. It contains the main method, initializes the core services, and launches the initial UI.	Methods: main(String[] args), initializeServices(), startHomeScreen().
Dashboard.java	Represents the main control panel or home screen of the application. It provides access to key functions like searching, booking, and managing customers.	Methods: displayMainView(), showAvailableRoomsTable(), handleSearchAction(), MapsToBookingForm(), handleExit().

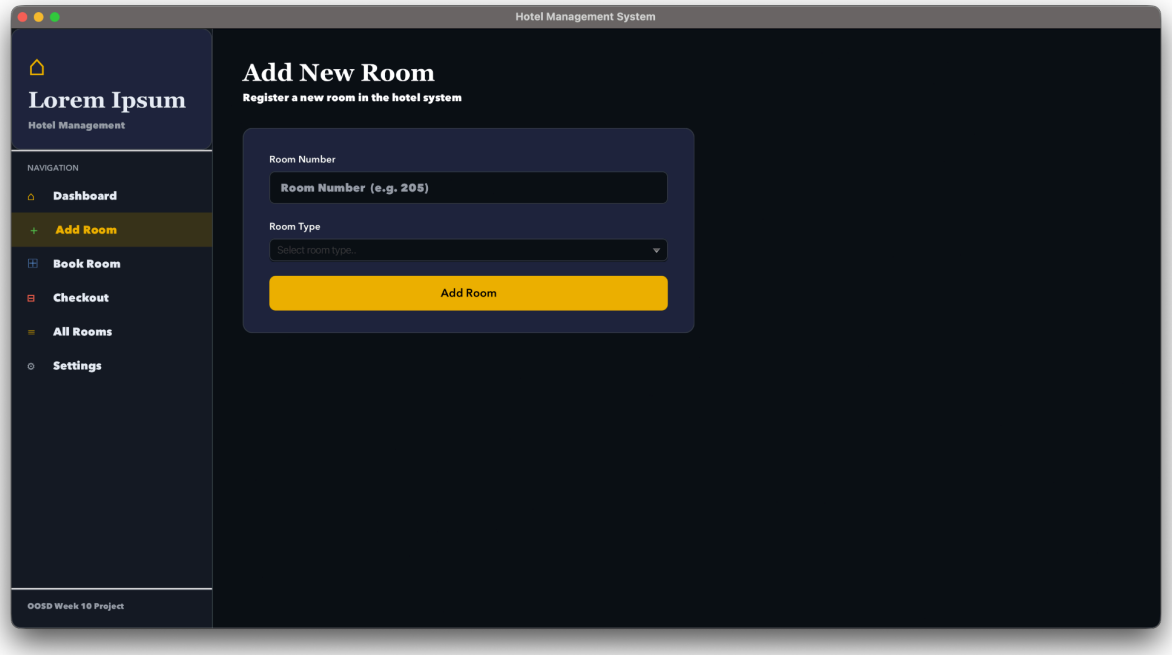
4. User Interface (UI) Design

4.1 Dashboard Page



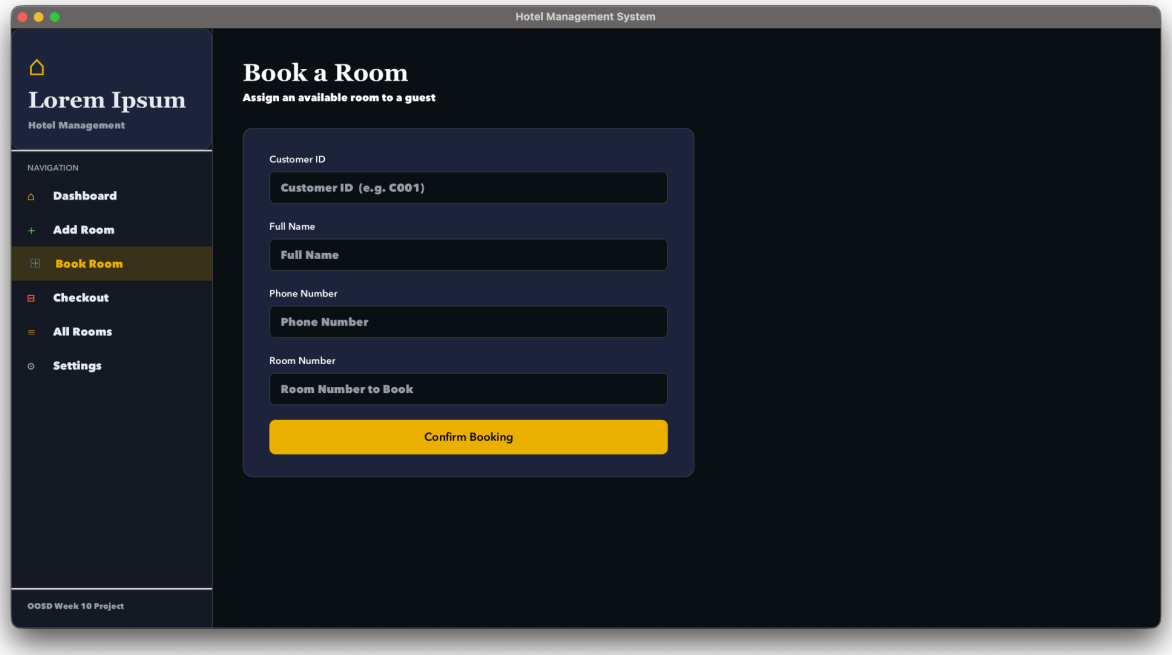
This is the main landing page, presenting live statistical cards, a functional clock, quick action navigation, and a dynamic "Available Rooms Preview" list. Behind the scenes, this view is populated by calling `getAvailableRooms()` and `getTotalRooms()` from the `HotelManager` to ensure real-time data accuracy, while a `Timeline` keeps the clock current.

4.2 Add Room Page



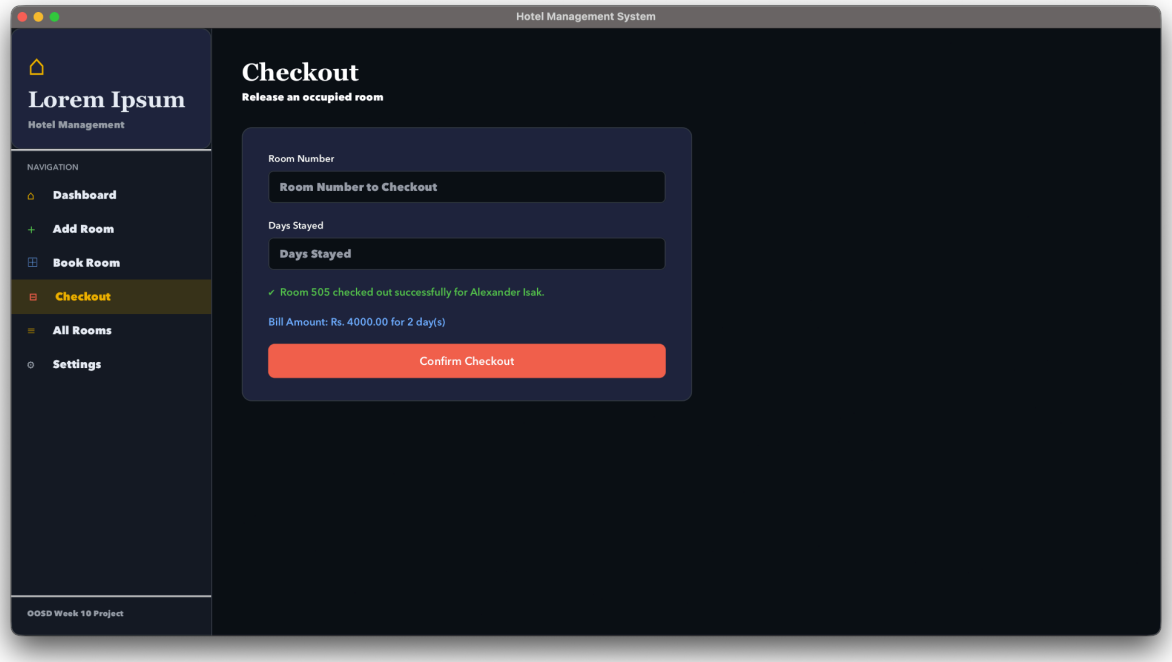
This dedicated form allows administrators to expand hotel capacity by inputting a room number into a text field and selecting a category from a dropdown menu. Upon submission, the system instantiates the corresponding room object (`StandardRoom` or `DeluxeRoom`) and utilizes the `manager.addRoom()` method to permanently save it to the inventory.

4.3 Book Room Page



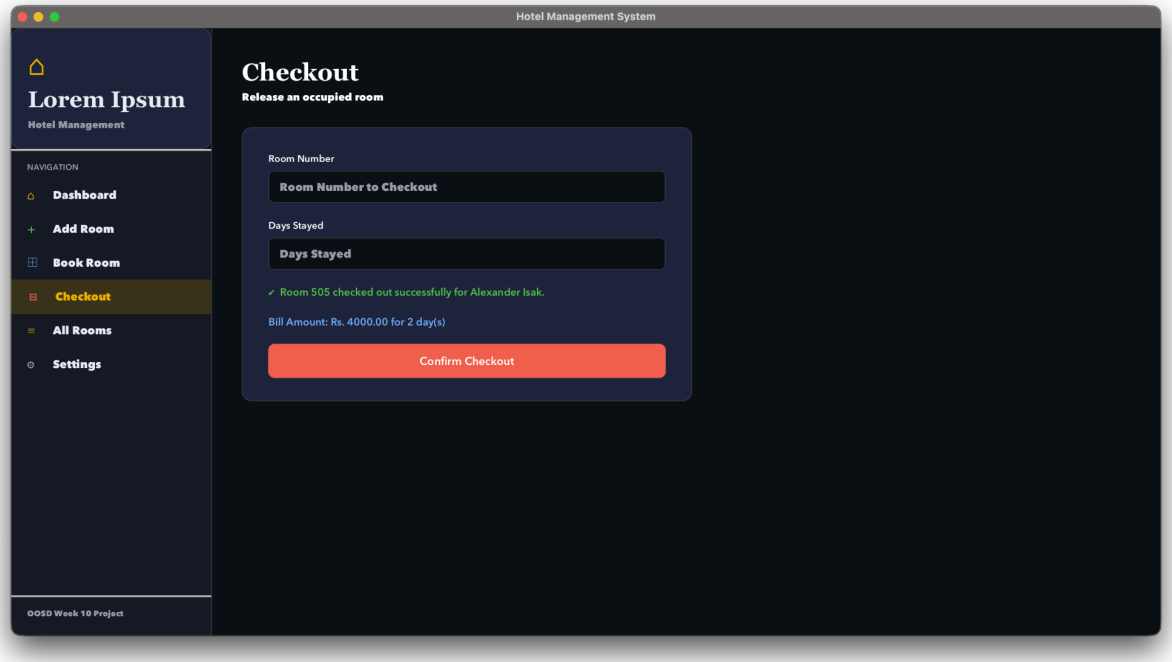
This form is the primary operational interface for checking in guests, requiring inputs for Customer ID, Full Name, Phone Number, and desired Room Number. When submitted, it triggers the synchronized `manager.bookRoom()` backend method to securely assign the guest and prevent double-booking, followed by automated visual feedback confirming the transaction.

4.4 Checkout Page



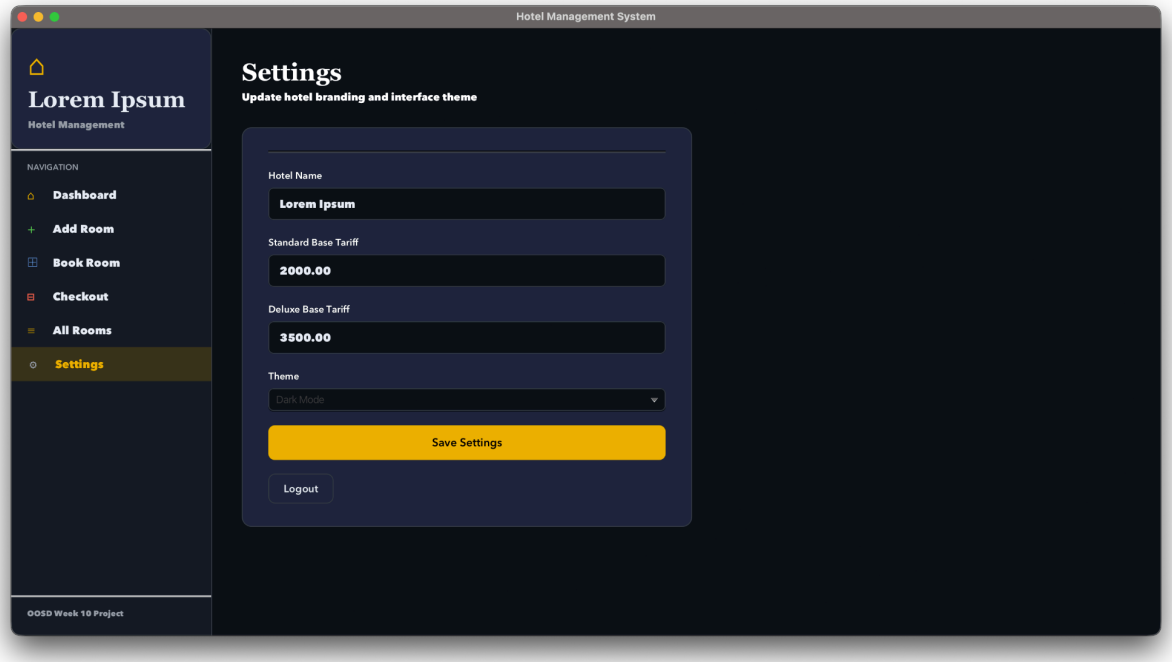
This streamlined form processes guest departures by requiring only a room number input, utilizing a distinct red confirmation button to signify a final action. Upon submission, the UI invokes the `manager.checkout()` method to instantly update the room's status to available and clear the associated customer record from the backend hash map.

4.5 All Rooms Page



This comprehensive inventory view presents a dynamic, alternating-color data grid displaying every registered room alongside its type, availability status, and current guest. It utilizes the `manager.getAllRooms()` backend method to fetch and populate the complete data set, featuring built-in filtering controls and a refresh button to help administrators easily sort, search, and monitor the hotel's total capacity.

4.6 Settings



The Settings page is a restricted administrative control center where managers can customize the hotel's display name, adjust base room tariffs, and toggle the application between Dark and Light UI themes. Access to this section is protected by an authentication wall, which intercepts unauthorized clicks and redirects the user to an Admin Login screen to verify their credentials. Behind the scenes, it utilizes `buildSettingsPage()` to construct the form, `manager.updateBaseTariffs()` to save new pricing to the backend, and `applyTheme()` to dynamically swap the application's color palette.

5. Conclusion

The Hotel Management System provides a practical and reliable way to handle daily hotel tasks by combining a strong Java backend with an easy-to-use JavaFX interface. By using core programming principles and thread-safe booking methods, the app ensures that guest records and room availability stay accurate and secure. This setup allows staff to quickly check guests in or out, manage different room types, and view live occupancy statistics from a single dashboard.

The inclusion of file-based data saving also means that important information is never lost between sessions, making the system a dependable tool for small to medium-sized hotels. Ultimately, this project demonstrates how modern software can simplify complex administrative work, providing a professional and efficient solution for better hotel management.